同济大学计算机系

操作系统课程实验报告

学　　号 2353761

姓　　名 王皙晶

专　　业 计算机科学与技术

授课老师 方钰

实验六：UNIX V6++中新进程创建与父子进程同步

一、实验目的

（1）在 UNIX V6++实验环境中编写使用父进程创建子进程的系统调用 fork、进程终止及父子进程同步的系统调用 exit 和 wait 的应用程序；
（2）进一步熟悉 UNIX V6++中关于进程创建、调度、终止和撤销的全过程；

二、实验结果

（1）完成实验 4.1～4.3，建立符合要求的进程树，并通过父进程是否执行 wait，执行几个 wait 来调整父子进程之前的同步顺序，实现父进程等待所有子进程、父进程先于所有子进程和父进程先于部分子进程等场景，截图展示程序运行结果；

1. 父进程等待所有子进程的场景：
ws=2，代表等待两个子进程都结束后再结束自己。

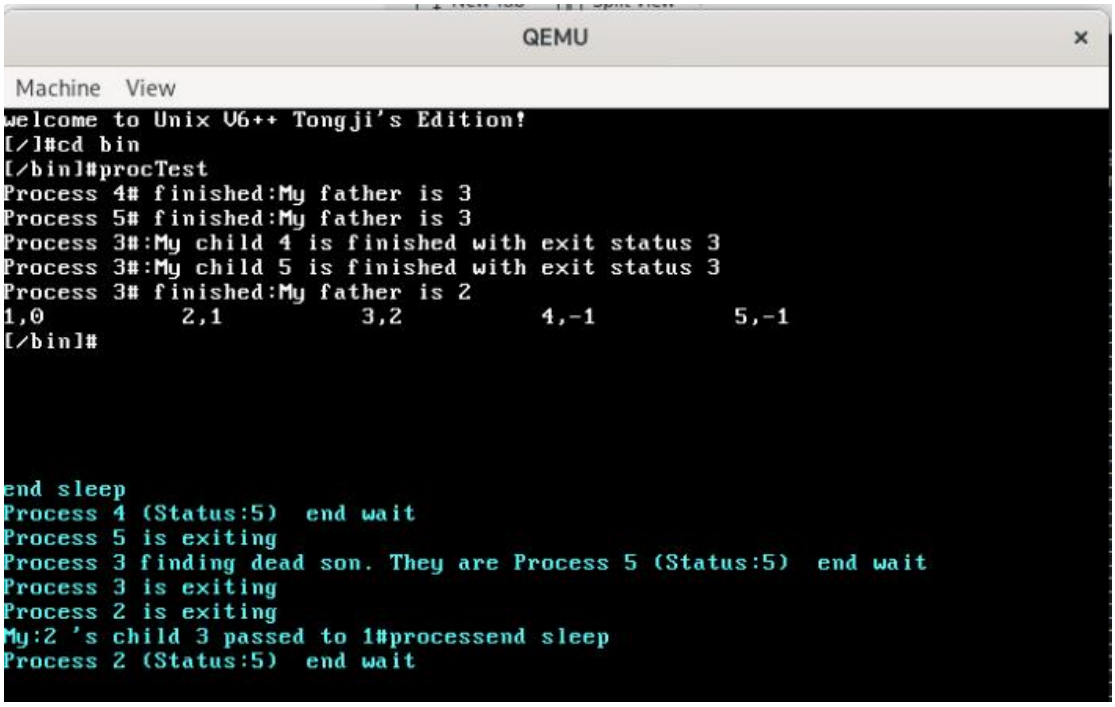

图 1

2. 父进程先于所有子进程的场景：
ws=0，代表上台后不下台，直接结束自己。

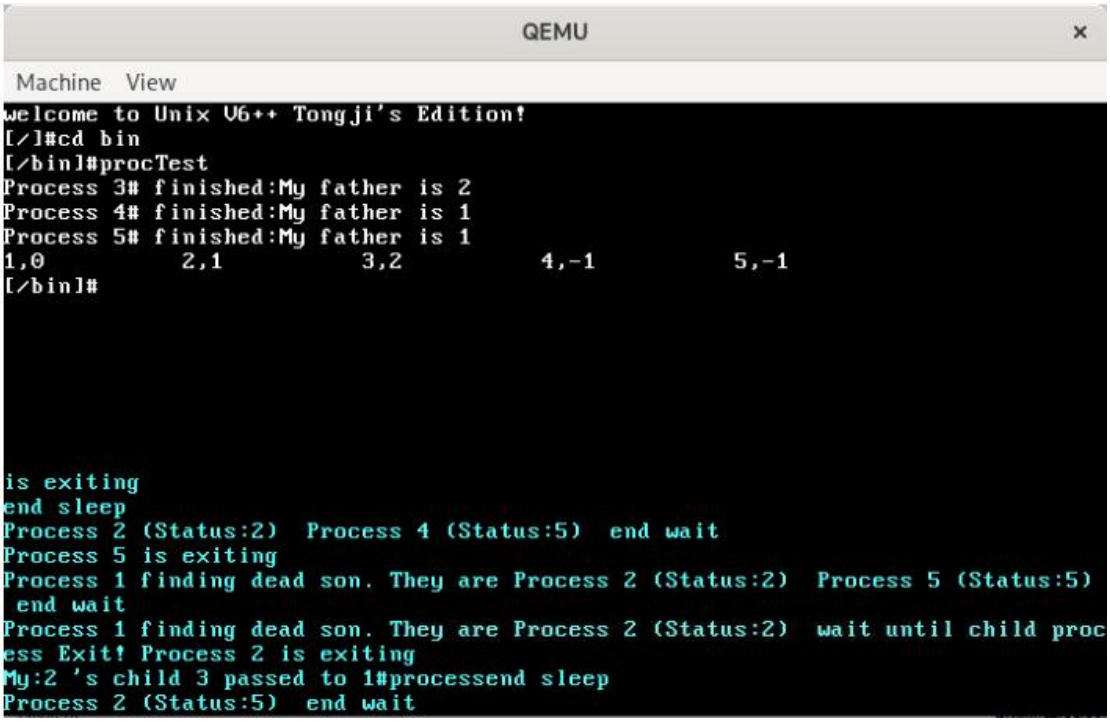

图 2

3. 父进程先于部分子进程的场景：

ws=1，代表等待一个子进程后再结束后自己。

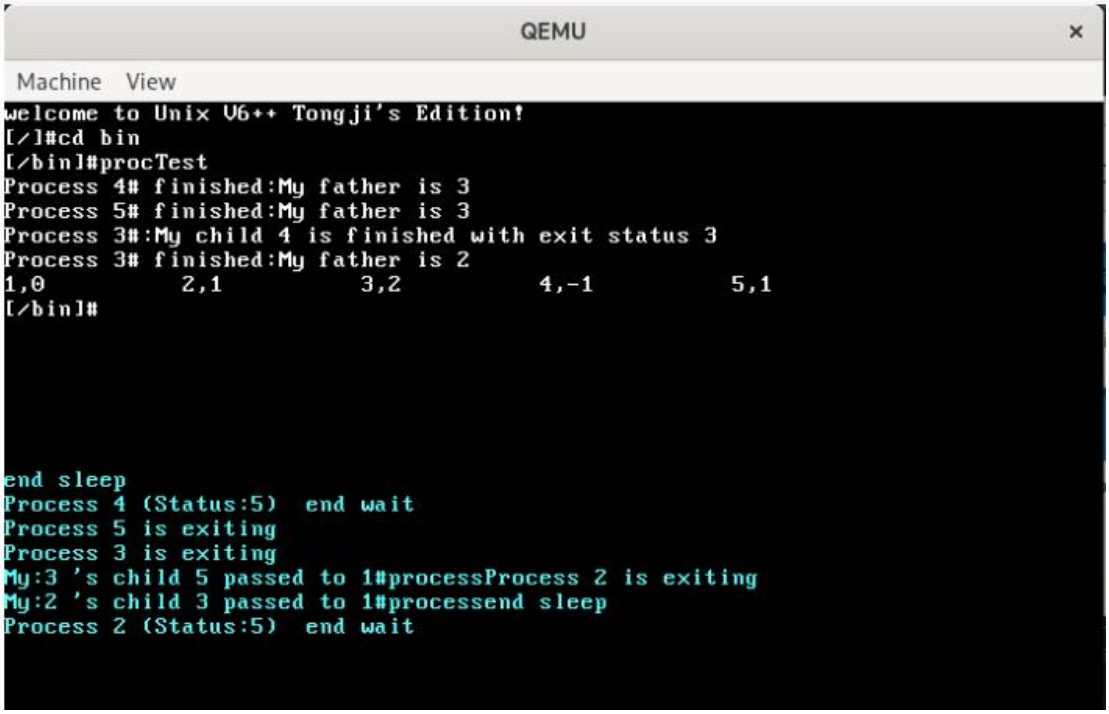

图 3

（2）针对实验 4.3，采用绘图或者文字方式解释实验报告中图 5 的输出，包括：进程的调度顺序及产生这样的调度顺序的原因，5#在最

后的打印输出语句时，为什么显示进程还存在，且父进程为 1#，并回答 5#将由谁在何时回收；

后文的示意图中只是标注了一些主要步骤，有一些细节有遗漏，因此再用文字解释一遍。先回答进程的调度顺序及产生这样调度顺序的原因：

2#进程执行 main1 主程序，fork 出 3#进程，通过 sleep 等待 2 秒钟后，打印输出当前系统中所有进程及其父进程 ID 号。**此时，4#已经终止并由 3#进程回收， 5#已经终止但唤醒 3#进程后，3#进程并没有再执行 wait 操作，因此没有回收 5#进程的 proc 结构，所以输出中 5#进程的父进程的进程号是 1#，并不是-1；而 3#虽然终止，但由于 2#没有执行 wait 回收，因而可以看到该进程还存在，且其父进程仍然为 2#；**

3#进程作为 2#进程的子进程，先后 fork 出 4#和 5#两个子进程（由于这两个 fork 中 3#进程作为父进程，均在内存创建成功子进程呢个，不曾入睡，所以优先数变化不大，因而重算时不会设置 RunRun，**使得 3#进程始终没有被抢占），并执行一个 wait 操作，处理部分（1 个）子进程的终止**。其中，在执行第一次 wait 时，因等待子进程终止而入睡，4#进程执行 exit 时将其唤醒，重新上台后回收 4#，而在返回用户时态前，由于优先数的变化被 5#进程抢占。再次上台后，执行 printf 后，通过 exit 结束自己，没有回收 5#剩余部分；

在 3#进程入睡后，对于此时同在就绪状态的 4#和 5#，作为刚刚创建好的子进程，具有相同的优先数 0，由于 UNIX V6++的 Swtch 程序优先选择 ID 号小的#进程，所以 4#先上台，在 fork 返回用户态前，由于重算优先数被 5#抢占。而 5#在 fork 返回用户态前，同样由于重算优先数被 4#抢占。

4#再次上台后，执行 printf 打印出自己的 ID 号和父进程的 ID 号后，结束自己，唤醒 3#，并向 3#发生终止码，终止码为父进程 ID 号。而 5#在 3#执行第一个 wait 返回用户时态前抢占 3#，执行 printf 打印自己的 ID 号和父进程的 ID 号后，结束自己，同样唤醒 3#，**但此时 3#处于就绪状态，唤醒操作并没有作用，并向 3#发生终止码，终止码为父进程 ID 号。但是 3#上台后没有执行其他 wait 操作，所以没有回收 5#。**3#下台后，2#从 sleep 中醒来，打印当前进程及其父进程。

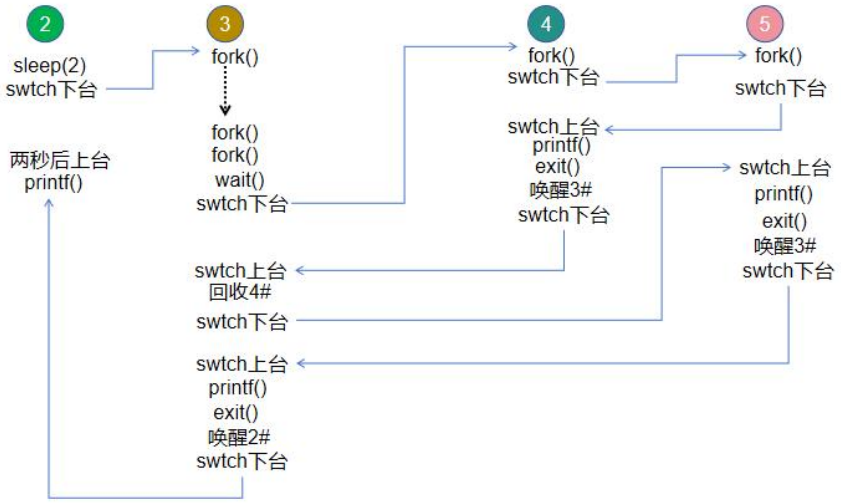

图 4

在上述解释的基础上，不难回答：“5#在最后的打印输出语句时，为什么显示进程还存在，且父进程为 1#，并回答 5#将由谁在何时回收”，答案就是因为 3#进程并没有回收 5#剩余的部分，因此 getppid 当然可以取到 proc 结构中的 p_ppid，从而显示出来。由于 3#进程并没有回收 5#进程，系统会将 3#尚未回收的子进程（即 5#进程）的父进程改为 1#进程，后续 5#进程将由 1#进程进行回收。

顺便再回答一下 4.3 中的另一个问题：“为什么 4#进程和 5#进程终止时，都能找到自己的父进程是 3#进程？”，答案是因为 4#和 5#进程终止时，父进程 3#还没有终止，因此当然可以找到自己的父进程是 3#进程了。（另一个问题就是题干中的问题，不再重复回答。）

（3）分别设计合理的方案，修改代码实现图 6 和图 7 的输出结果，采用绘图或者文字方式解释输出，包括：进程的调度顺序及产生这样的调度顺序的原因。

1.采用如下代码，实现实验指导中的图 6：

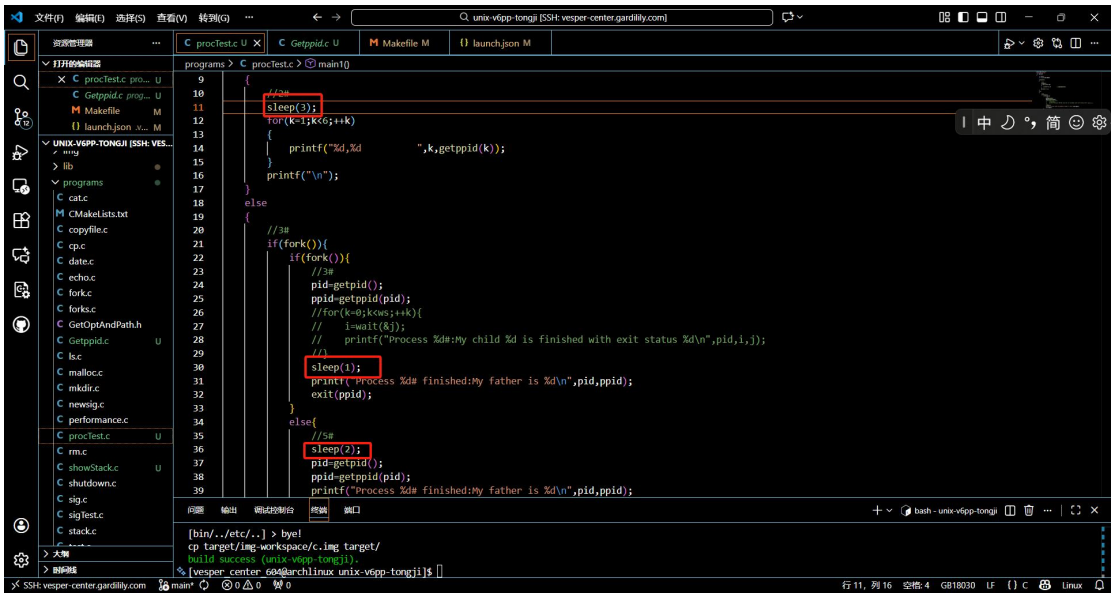

图 5

示意图和前文图 x 类似，在此不再呈现，只做文字解释：上述代码之所以能实现实验指导中图 6 的输出效果是由于：2#由于 sleep(3)，被 swtch 下台，3#进程上台，fork 出了 4#和 5#进程。由于**这两个 fork 中 3#进程作为父进程，均在内存创建成功子进程，不曾入睡，所以优先数变化不大，因而重算时不会设置 RunRun，使得 3#进程始终没有被抢占**，3#进程执行到即将要 printf 时，sleep(1)，入睡，此时 3#也被 swtch 下台，换新建立的 4#进程上台，4#进程上台后，重算了 pri 值，被 swtch 下台，换 5#进程上台，但是 5#进程一上台就被 sleep(2)入睡，又被 swtch 下台，4#上台继续执行，输出自己的 printf（图 6 的第一行输出），然后 exit 自己，此时 5#进程还没有醒来，因此 3#被 swtch 上台，继续执行自己的 printf（图 6 的第二行输出），并 exit 自己。由于 3#进程没有进行 wait 操作，所以 4#的 proc 也是由 1#进程回收的（这个根据下面程序运行结果可以得到），

3#进程 exit 后，把当前未终止的子进程 5#传给 1#进程，然后 5#进程被调度上台，后续输出自己的 printf（图 6 的第三行输出）并 exit 自己。后续由 1#进程完成对 5#进程 proc 的回收。由于设计的 sleep 时间差，2#进程输出时（图 6 的第四行），1#进程已经回收了 4#和 5#的 proc 块，所以打印出来的 4#、5#的父进程是-1（代表 4#、5#已被回收）。

程序运行结果：

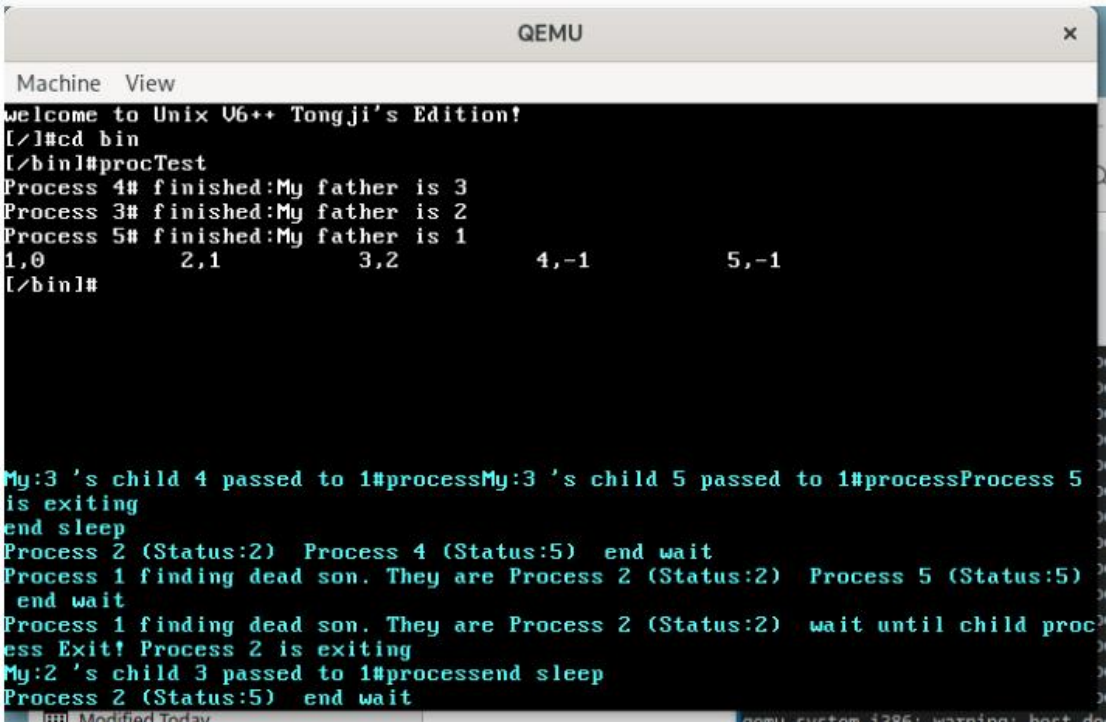

图 6

2.采用如下代码，实现实验指导中的图 7：

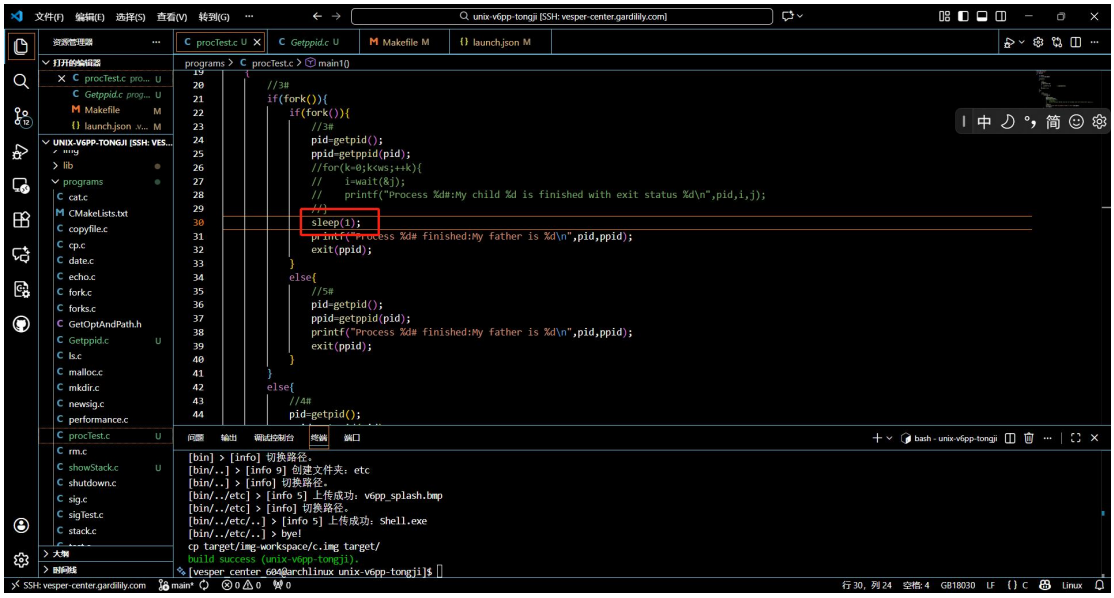

图 7

示意图和前文图 x 类似，在此不再呈现，只做文字解释：上述代码之所以能实现实验指导中图 7 的输出效果是由于：2#由于 sleep(2)，被 swtch 下台，3#进程上台，fork 出了 4#和 5#进程。由于**这两个 fork 中 3#进程作为父进程，均在内存创建成功子进程，不曾入睡，所以优先数变化不大，因而重算时不会设置 RunRun，使得 3#进程始终没有被抢占，**3#进程执行到即将要 printf 时，sleep(1), 入睡，此时 3#也被 swtch 下台，换新建立的 4#进程上台，4#进程上台后，重算了 pri 值，被 swtch 下台，换 5#进程上台，但是 5#进程上台后，重算了 pri 值，被 swtch 下台，又换 4#进程上台。4#上台继续执行，输出自己的 printf（图 7 的第一行输出），然后 exit 自己，此时 3#进程还没有醒来，因此 5#被 swtch 上台，继续执行自己的 printf（图 7 的第二行输出），并 exit 自己。由于 3#进程没有进行 wait 操作，所以 4#、5#的 proc 也是由 1#进程回收的（这个根据下面程序运行结果可以得到），3#进程上台后续输出自己的 printf（图 6 的第三行输出）并 exit 自己。后续由 1#进程完成对 4#、5#进程 proc 的回收。由于 2#进程输出时（图 6 的第四行），1#进程还没有回收 4#和 5#的 proc 块，所以打印出来的 4#、5#的父进程是 1（代表 4#、5#还未被完全回收）。

程序运行结果：

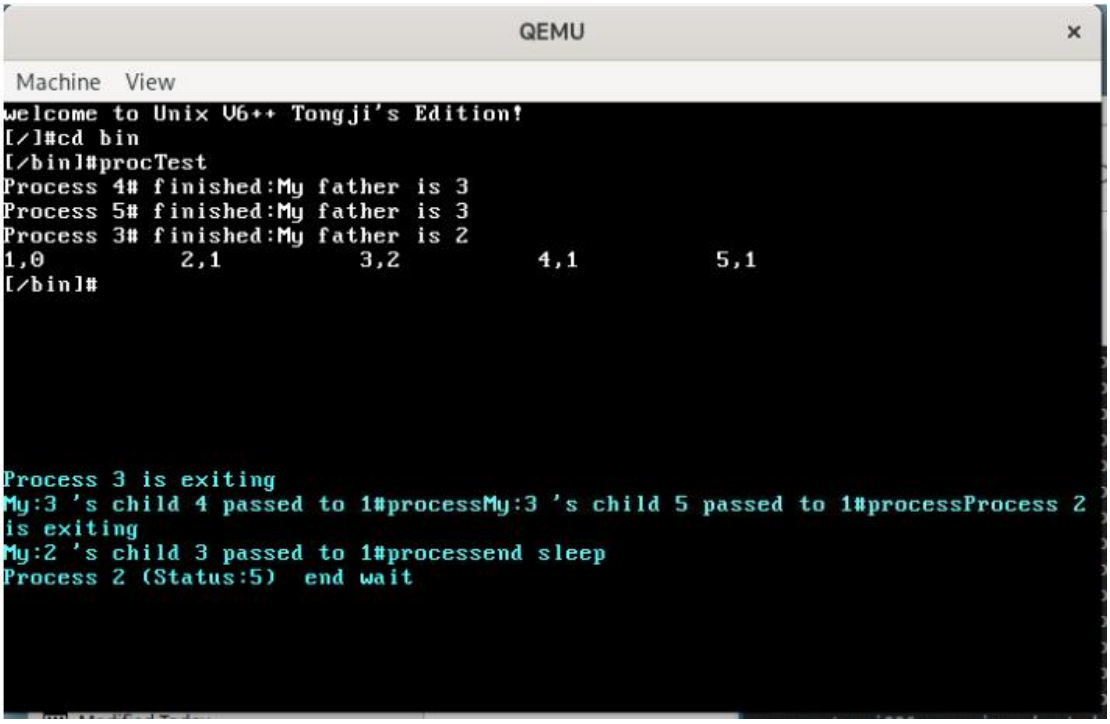

图 8